

Temporal Visual Reuse in OpenVLA-OFT: A Sequential Negative Study

Ved Dwivedi, Cheng Yang, and Bo Yuan

Abstract—Temporal redundancy may allow a vision–language–action (VLA) policy to reuse visual computation across control queries. We tested this idea with one pinned OpenVLA-OFT checkpoint on LIBERO. The study developed five methods in sequence: state-aware whole-prefix refresh (SAVR), camera-asymmetric refresh (ACR), layer- and tile-selective key–value reuse (BRACE), intervention-trained risk routing (PAIR), and residual cache-action correction (CAC). No method passed both its reliability and efficiency gates. Nine SAVR settings obtained 0–52 successes per 100 episodes at skip rates of 34.69–83.68%. A conservative SAVR redesign obtained 69/70 successes but skipped only 0.95% of visual refreshes. ACR matched its batched full-refresh baseline at 67/70 successes and reused the scene camera on 25.24% of queries, but did not reduce complete query time. BRACE and PAIR showed timing headroom. Their action-quality results are provisional because a later audit found an action-readout mismatch. After correction, the recursive D62 cache retained 22.60% gross timing headroom but obtained 0/120 successes. Dense inference obtained 68/120. These results apply to the tested cache semantics and system. They do not reject current-frame compression or cache-aware training.

Index Terms—Vision–language–action models, robot learning, adaptive computation, visual caching, closed-loop reliability.

I. INTRODUCTION

VISION–LANGUAGE–ACTION (VLA) policies map images, language instructions, and robot state directly to actions. RT-1 and RT-2 established large-scale language-conditioned robotic control and transfer from vision–language pretraining [1], [2]. OpenVLA released a 7B-parameter open VLA, and OpenVLA-OFT combined continuous action prediction, parallel decoding, and action chunking to improve both task success and action-generation throughput [3], [4]. These capabilities make repeated visual and transformer computation a practical deployment cost.

Adjacent observations in manipulation are often visually similar. This motivates reusing projected visual tokens or internal key–value (K/V) states instead of recomputing them at every policy query. Existing work demonstrates several successful variants of this idea: VLA-Cache selectively reuses temporally stable tokens, EfficientVLA combines multiple pruning and caching mechanisms, DepthCache uses geometric structure, and learned adaptive caching trains task-aware token-selection modules [5]–[8]. These results establish that temporal redundancy can be useful. They do not imply that

every cache boundary, controller, model, or closed-loop operating point is reliable.

We ask whether a frozen VLA can reuse stale visual computation while preserving terminal task success and reducing complete inference time. We study this question through a sequence of methods. State-Aware Visual Refresh (SAVR) reused the full projected prefix from both cameras. Its controller used image, state, action, and cache-age signals. Asymmetric Camera Refresh (ACR) kept the wrist image current and reused only the scene-camera block. BRACE applied reuse to selected layer, camera, and tile K/V groups. PAIR learned a risk score from controlled stale-state interventions. Cache Action Correction (CAC) proposed a small residual adapter that would correct the cached action.

Several system components worked as intended. Visual work could be removed. Routing overhead could be reduced. The proposed CAC adapter fit within the memory and timing budgets. However, no method passed both its reliability and efficiency criteria. An independent audit also found that the custom BRACE, PAIR, and early CAC helper read the 56 action states one token later than the official evaluator. The BRACE and PAIR action-quality results are therefore provisional. We corrected the readout and obtained exact agreement with the released evaluator. The corrected D62 cache then failed all 120 closed-loop episodes across four LIBERO suites.

The paper makes four contributions:

- 1) It reports a documented progression from whole-prefix reuse to camera-, tile-, router-, and correction-level designs, including failed operating points rather than only the final candidate.
- 2) It separates *systems headroom* from *closed-loop reliability*, showing why skipped operations and microbenchmark speedups are insufficient evidence for an embodied policy.
- 3) It reports an independent semantic requalification and explains why checks sharing one helper cannot authenticate agreement with the official policy.
- 4) It identifies design constraints supported across stages: camera-specific dynamics, induced-distribution shift, optimized dense baselines, exact cache provenance, and early closed-loop testing.

In this paper, reliability means terminal task success or a stated action-equivalence test. We did not measure collisions, constraint violations, or formal robot safety. The stages used different populations and were developed in sequence. Cross-stage figures are descriptive. We do not treat them as pooled causal estimates.

Ved Dwivedi is with John P. Stevens High School, Edison, NJ, USA.

Cheng Yang and Bo Yuan are with the Department of Electrical and Computer Engineering, Rutgers University–New Brunswick, Piscataway, NJ, USA.

Author-review manuscript, 2 September 2026. A double-anonymous submission must remove author-identifying information.

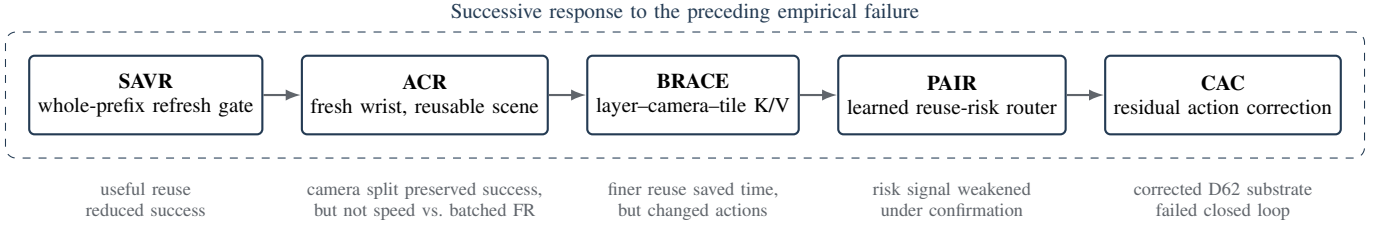


Fig. 1. The research sequence. Each redesign changes the abstraction used to manage stale visual computation. The sequence is a retrospective synthesis of staged experiments, not a claim that one preregistered protocol specified all five methods in advance.

II. RELATED WORK

A. VLA policies and evaluation

OpenVLA combines a Llama-family language backbone with fused DINOv2 and SigLIP visual features and exposes a public VLA training and inference stack [3]. OpenVLA-OFT replaces discrete action-token generation with a continuous action head and executes action chunks, providing the model family and released evaluator used in this study [4]. LIBERO supplies procedurally generated manipulation tasks grouped into Spatial, Object, Goal, and LIBERO-10 suites [9]. Our experiments use one pinned four-suite OpenVLA-OFT checkpoint and the official terminal success criterion.

B. Efficient VLA inference

VLA efficiency can be improved at several model boundaries. EfficientVLA removes language layers, selects visual tokens, and caches features in an iterative action head [6]. VLA-Cache reuses K/V entries for visual tokens that change little and adjusts reuse by decoder layer [5]. DepthCache uses depth and motion to merge visual tokens [7]. Learned adaptive caching trains a token selector and a reuse-ratio predictor with the task objective [8]. Neural-introspection gating uses policy confidence to invalidate reuse when prediction becomes uncertain [10]. Action-JND learns action-conditioned tolerances for token compression, including stale K/V reuse [11]. AC²-VLA uses action context to route temporal reuse, token pruning, and component execution [12]. These methods use different models, training procedures, cache boundaries, and evaluations. We cite them to define the relevant design space. We do not use their published numbers as direct baselines.

Learned correction is also relevant to CAC. A2C2 corrects stale actions within an action chunk by using the latest observation and the base action [13]. Latent Bridge predicts feature or K/V changes between slow-backbone calls and uses on-policy data aggregation [14]. CAC differs from both. It was designed to predict the dense action effect after selective, recursively mixed-age K/V reuse. It also uses the physical source of each reused group as input. Residual action correction itself is not new. The proposed contribution was this specific cache-error formulation and test.

C. Closed-loop distribution shift

Offline calibration observes states generated by a reference policy. Once a modified policy chooses a different action,

it can induce a different future state distribution. This mismatch is fundamental in sequential imitation learning and can compound with horizon [15]. For a dense policy π_F and a reuse policy π_R , an offline replay samples $s \sim d_{\pi_F}$, whereas deployment evaluates $s \sim d_{\pi_R}$. In general,

$$d_{\pi_R}(s) \neq d_{\pi_F}(s). \quad (1)$$

This distinction motivates our separation between offline action diagnostics and closed-loop terminal outcomes.

III. PROBLEM FORMULATION

A. Dense policy and action chunk

Let q index policy queries, not simulator frames. At query q , the model receives instruction x , scene image o_q^s , wrist image o_q^w , and current proprioception p_q . The visual towers and projector produce ordered visual tokens

$$V_q = \mathcal{V}_\theta(o_q^s, o_q^w). \quad (2)$$

The fixed downstream transformer $\mathcal{T}_\theta$ and continuous action head h_θ produce an eight-step, seven-dimensional action chunk

$$A_q^F = h_\theta(\mathcal{T}_\theta(x, V_q, p_q)) \in \mathbb{R}^{8 \times 7}. \quad (3)$$

The evaluator executes all eight actions before the next normal query. A method can therefore change multiple environment transitions before observing the consequences of one cache decision.

B. Temporal cache and source provenance

At decoder layer ℓ and visual group g , let $K_{q,\ell,g}$ and $U_{q,\ell,g}$ denote current K/V entries. A binary refresh mask $M_{q,\ell,g} = 1$ selects current values. A value of zero reuses entries from physical source query $\sigma(q, \ell, g) < q$:

$$\tilde{K}_{q,\ell,g} = M_{q,\ell,g}K_{q,\ell,g} + (1 - M_{q,\ell,g})K_{\sigma(q,\ell,g),\ell,g}, \quad (4)$$

$$\tilde{U}_{q,\ell,g} = M_{q,\ell,g}U_{q,\ell,g} + (1 - M_{q,\ell,g})U_{\sigma(q,\ell,g),\ell,g}. \quad (5)$$

The physical age is $a_{q,\ell,g} = q - \sigma(q, \ell, g)$. Recursive reuse creates a mixed-age state because $a_{q,\ell,g}$ can vary by layer, camera, tile, and query. The resulting cached action is A_q^R .

TABLE I
CORE NOTATION.

Symbol	Meaning
q	Model-query index. Eight actions are normally executed per query
V_q	Current projected scene and wrist visual tokens
$M_{q,\ell,g}$	Refresh indicator for layer ℓ , group g
$\sigma(q, \ell, g)$	Physical source query of a reused K/V group
$a_{q,\ell,g}$	Source age $q - \sigma(q, \ell, g)$
A_q^F, A_q^R	Dense and reuse-path action chunks
$\hat{S}(m)$	Empirical terminal success of method m
$G_T(m)$	Complete-query time reduction relative to dense inference

C. Reliability and efficiency estimands

For a fixed population of N episodes, terminal success is

$$\hat{S}(m) = \frac{1}{N} \sum_{i=1}^N \mathbb{1}\{\text{episode } i \text{ succeeds under } m\}. \quad (6)$$

We report exact two-sided 95% binomial intervals where success counts are central [16]. These intervals are descriptive under an exchangeable Bernoulli model. They do not account for clustering by task or initial state. For paired dense and reuse policies, the reliability difference is $\Delta_S = \hat{S}(R) - \hat{S}(F)$.

Efficiency is evaluated at the complete policy-query boundary whenever a valid timing experiment exists:

$$G_T(m) = 1 - \frac{T_m}{T_F}, \quad (7)$$

where T_m includes the declared controller, cache, provenance, synchronization, and data-movement costs. Component CUDA time and logical reuse are mechanism measurements, not substitutes for G_T .

IV. SEQUENTIAL METHOD DEVELOPMENT

Table II summarizes the hypothesis tested at each stage. Each method was designed after the previous result was available. Later methods are therefore not independent confirmations of earlier choices.

A. SAVR: State-aware whole-prefix refresh

SAVR stores the visual prefix from the last refresh query τ_q . It computes image change d_q^{img} , normalized robot-state change d_q^{state} , recent action change d_q^{act} , and cache age $h_q = q - \tau_q$. SAVR refreshes when any signal exceeds its threshold:

$$r_q = \mathbb{1}[d_q^{\text{img}} > \gamma_I \vee d_q^{\text{state}} > \gamma_S \vee d_q^{\text{act}} > \gamma_A \vee h_q \geq H]. \quad (8)$$

If $r_q = 0$, the model receives V_{τ_q} instead of V_q . SAVR1 uses an average over scene and wrist image changes. SAVR2 replaces that average with separate local-patch camera gates, groups state and action coordinates semantically, forbids consecutive reuse, and adds gripper-transition and prefix-budget vetoes. SAVR3 changes the wrist threshold and adds a translation-direction-reversal veto. SAVR3 is explicitly post-hoc and was evaluated once on a held-back development subset.

B. ACR: Camera-asymmetric refresh

SAVR diagnostics showed that the wrist image changed while the scene image remained stable. ACR therefore forms

$$\tilde{V}_q^{\text{ACR}} = \begin{bmatrix} V_{\tau_q}^s; V_q^w \end{bmatrix}, \quad (9)$$

where the wrist block is always current and only the scene block can be reused. The first dual-path implementation executed the camera paths separately. A later implementation processed both cameras as one batch on refresh queries. This introduced batched full refresh (BFR) as the correct dense reference. An offline V4 audit then showed that a horizon of two still allowed two consecutive reuses. V5 corrected this error with a controller latch that required a refresh after every reuse. Its offline gate passed. The intended CUDA-graph executor did not complete a valid performance experiment.

C. BRACE: Layer-camera-tile K/V reuse

BRACE implements (5) on 4×4 spatial tiles at decoder onset layers $\ell \in \{2, 6, 9, 11\}$. Profiles vary scene and wrist reuse budgets and cap scene/wrist age at four/two queries. A source ledger stores $\sigma(q, \ell, g)$ for every group. The design tests whether coarse camera decisions fail only because they do not isolate action-relevant regions.

D. PAIR: Intervention-trained reuse-risk routing

PAIR uses controlled substitutions of the actual stale K/V values that a recursive cache would deploy. For expert action A_q^* and action loss L , the intervention target is excess expert imitation loss

$$R_q = L(A_q^R, A_q^*) - L(A_q^F, A_q^*). \quad (10)$$

A compact router $f_\psi(z_q)$ predicts reuse risk from current features and exact provenance. Its purpose is not merely to minimize mean error: the predeclared reliability endpoint evaluates the worst decile through

$$\text{CVaR}_{0.9}(R) = \mathbb{E}[R \mid R \geq Q_{0.9}(R)], \quad (11)$$

at matched service. Recursive contracts of horizons one, two, and four carry the real mixed cache forward rather than treating token substitutions independently.

E. CAC: Residual correction after reuse

CAC keeps the fixed D62 recursive cache and adds a small adapter while the VLA backbone remains frozen. The current visual representation has 512 projected tokens of width 4096. Pooling a 4×4 grid for each camera produces $C_q \in \mathbb{R}^{32 \times 4096}$. Source differences from four decoder onset layers produce 128 current-minus-source tokens D_q . The 56 cached-path action states are grouped by action step to form $Z_q^R \in \mathbb{R}^{8 \times 4096}$. The adapter also receives exact source provenance Π_q , current proprioception p_q , and instruction embedding e_x . It predicts

$$\Delta A_q = g_\phi(Z_q^R, C_q, D_q, \Pi_q, p_q, e_x), \quad (12)$$

$$\hat{A}_q^{\text{CAC}} = A_q^R + \Delta A_q. \quad (13)$$

TABLE II
METHOD PROGRESSION AND THE SPECIFIC HYPOTHESIS TESTED AT EACH CACHE BOUNDARY.

Program	Reused object	Current information retained	Decision or correction rule	Primary unresolved risk
SAVR	Complete projected two-camera prefix	Current proprioception and downstream execution	Thresholds on image, state, recent action, and cache age	Both cameras become stale together. Offline calibration can differ from online behavior
ACR	Scene-camera projected block	Wrist camera, proprioception, and downstream execution	Camera-specific gate. Later versions isolate reuse and require a refresh	Split execution overhead can erase visual savings
BRACE	Selected K/V groups by layer, camera, and tile	Refreshed groups and exact source ledger	L1/cosine change, semantic sidecar, bounded source age	Fine-grained masks can still change the action
PAIR	Same BRACE K/V groups	Same as BRACE	Learned risk score from controlled stale-state interventions	Offline regret ranking may not generalize or predict terminal success
CAC	Same D62 mixed-age K/V substrate	Current visual summaries, proprioception, instruction, and provenance	Residual adapter predicts $A_q^F - A_q^R$	Low-dimensional correction may be unable to repair a severely degraded substrate

The full adapter projects the inputs to width 256 and uses two cross-attention blocks. It has 5,070,599 trainable parameters. It does not reconstruct fresh K/V states and does not call the dense policy. Before training, D62 had to pass a closed-loop screen. Dense success had to be at least 75%. D62 success had to be at least 50%. The dense-minus-D62 gap could not exceed 35 pp. These gates prevent training a small residual model on a cache policy that has no demonstrated control ability.

V. EXPERIMENTAL DESIGN AND EVIDENCE STATUS

A. Pinned system

All reported experiments use the released OpenVLA-OFT stack, checkpoint `openvla-7b-oft-libero-four-suite`, two RGB cameras, current proprioception, and the official LIBERO terminal success function. Runs use at most one NVIDIA TITAN RTX with a strict project memory ceiling below 23 GiB. Configurations, query budgets, populations, and stopping gates were frozen before the corresponding protected outcomes were opened.

The study is not one randomized experiment. SAVR used LIBERO-Spatial development populations. ACR used LIBERO-Object development populations. BRACE and PAIR used outcome-blind model queries or offline intervention records. C1H used 30 conditions from each of the four suites. We report each stage separately and do not pool success estimates.

B. Technical stop versus scientific result

A *technical stop* occurs before the declared measurement is valid, for example because of a loader, memory, schema, or CUDA-graph error. Such a run provides engineering evidence only. A *scientific stop* occurs after the complete frozen population and validity checks pass but the method gate fails. Failed episodes remain in the estimate.

C. Semantic requalification

On 31 August 2026, an independent comparison with the released evaluator found that the shared BRACE/PAIR/CAC

helper selected hidden states $[-57 : -1]$, whereas the official continuous regression head consumed $[-58 : -2]$. The shift omitted the official first action state and added a state outside the official slice. Internal branches agreed because they shared the same helper.

All historical artifacts were preserved. Official-policy-dependent BRACE and PAIR interpretations are marked provisional. Before corrected CAC evaluation, one structurally derived action-readout contract was compared with an independent official oracle. Hidden states, normalized actions, and unnormalized actions then matched with maximum absolute error zero. This corrected boundary is used for the C1 systems result and C1H closed-loop result.

VI. RESULTS

A. SAVR: Useful whole-prefix reuse reduced success

Full refresh succeeded on all 100 SAVR calibration episodes. Table IV summarizes the complete SAVR1 range and the reliability-constrained redesigns. Across nine SAVR1 settings, online skip ranged from 34.69% to 83.68%, while success ranged from 52/100 to 0/100. The highest-success SAVR1 setting was 48 pp below full refresh.

SAVR2 recovered 30/30 success only when it never reused. At 6.72% and 10.57% skip, it obtained 29/30 and 27/30. SAVR3 obtained 69/70 success but reused only 9 of 944 policy queries (0.9534%). It failed both its exact-success and minimum-5%-skip gates.

The failure appeared immediately. Among 87 comparable SAVR1 episodes, the first reused query changed the predicted action hash in all 87. Offline full-refresh replay underestimated online reuse for every SAVR1 setting. Within the nominal 25% family, online overshoot increased from 9.71 pp at horizon two to 22.30 pp at horizon eight. In addition, 374 of 783 reuse decisions hid at least one camera threshold exceedance behind the two-camera average. Of these cases, 372 were wrist-only.

The best SAVR1 setting also showed an association between early reuse and failure. Success was 7/23 when first reuse occurred by query two, 18/42 at queries three or four, 24/32 at query five or later, and 3/3 when reuse never occurred. Every failed episode reached two consecutive reuse queries.

TABLE III

PROGRAM-WIDE EVIDENCE LEDGER. COUNTS REFER TO COMPLETED RECORDS USED FOR THE STATED RESULT. IMPLEMENTATION-ONLY TESTS AND INVALID TECHNICAL RUNS ARE EXCLUDED.

Stage	Evaluation type	Completed evidence	Main finding	Status
FR timing pilot	Closed loop and component timing	50 episodes, 662 steady queries	49/50 success. Visual backbone and projector used 15.874% of query CUDA time	Feasibility pass
SAVR1–3	Closed-loop development	1,160 primary episodes	No tested whole-prefix setting passed both success and reuse gates	Scientific stop
ACR A5	Closed-loop development	90 episodes	Best candidate obtained 29/30 success at 26.06% scene reuse	Scientific stop
ACR V3-D	Paired closed loop and timing	140 episodes	ACR and BFR both obtained 67/70 success. ACR was not faster than BFR	Scientific stop
ACR V5	Outcome-blind replay and executor work	100 traces, 1,773 queries	Offline reuse gate passed. GPU executor never produced a valid performance record	Technical stop
BRACE B3	Outcome-blind physical timing and action checks	356 model queries	P2-D50 reduced complete-cycle time by 12.23%. All tested cached actions changed	Provisional
PAIR P3R/P4/P4B	Timing and offline intervention contracts	210 P3R calls, 40 P4 contracts, 1,784 P4B calls	Timing gate passed. The P4 risk signal did not confirm on the larger population	Provisional stop
Corrected CAC C1/CIH	Systems qualification and paired closed loop	97 C1 calls, 240 C1H episodes, 8,525 C1H queries	Adapter fit the system budget. Corrected D62 obtained 0/120 success	Scientific stop

TABLE IV

REPRESENTATIVE SAVR RESULTS. INTERVALS ARE EXACT TWO-SIDED 95% BINOMIAL INTERVALS. STAGES USE DIFFERENT DEVELOPMENT POPULATIONS.

Method	Success	95% CI (%)	Skip (%)	Gate
FR	100/100	96.38–100	0.00	Reference
SAVR1 best	52/100	41.78–62.10	34.69	Fail
SAVR1 max skip	0/100	0.00–3.62	83.68	Fail
SAVR2-b05	30/30	88.43–100	0.00	Fail skip
SAVR2-b10	29/30	82.78–99.92	6.72	Fail success
SAVR2-b15	27/30	73.47–97.89	10.57	Fail success
SAVR3	69/70	92.30–99.96	0.95	Fail both

TABLE V

FULL-REFRESH TIMING PILOT. THE PILOT CONTAINED 50 EPISODES, 49 SUCCESSES, AND 662 STEADY-STATE POLICY QUERIES.

Component	Mean CUDA time (ms)	Query share (%)
Visual backbone	191.12	15.08
Visual projector	10.08	0.80
Action head	5.03	0.40
Residual downstream path	1061.21	83.73
Complete query	1267.44	100.00

These counts are exploratory. They do not establish that early or consecutive reuse caused the failure.

SAVR3 suppressed reuse through overlapping vetoes. Its log contained 279 wrist-image triggers, eight full-image triggers, 899 stable-fresh decisions, 424 prefix-budget triggers, and 412 translation-reversal triggers. The counts overlap because one query can activate several conditions. Periodic-refresh and visual-only baselines were not opened after the whole-prefix program failed its development gate. The final SAVR holdout remained sealed.

The separate timing pilot established the size of the opportunity before SAVR evaluation. The visual backbone and projector used 201.19 ms per query, or 15.874% of measured query CUDA time. Even a zero-overhead method that removed all of this work had a speedup ceiling of $1.189\times$. Mean observation preprocessing was 17.59 ms in wall time. This pilot is not part of the 1,160 primary SAVR evaluation episodes.

B. ACR: Camera asymmetry helped reliability, not incremental speed

The first ACR screen evaluated three 30-episode candidates. All exceeded 15% scene reuse, but none achieved the required 30/30 overall and 3/3 per-task success. The conservative candidate achieved 29/30 at 26.06% scene reuse. The two more permissive candidates achieved 24/30 and 23/30 at 47.40% and 49.44% reuse. Their exact 95% success intervals were 82.78–99.92%, 61.43–92.29%, and 57.72–90.07%.

ACR V2 showed why visual work is not wall time. Reuse reduced median visual CUDA time from 150.566 to 75.104 ms. The weighted dual path was nevertheless 41.03% slower than full refresh. The implementation still placed audit work on the critical path. V3 removed that work and batched the two-camera refresh. A 64-query mechanism test passed exact action and token checks and predicted a 31.41% visual CUDA reduction at the frozen reuse weight.

The subsequent paired closed-loop evaluation is the decisive V3 result. BFR and ACR each achieved 67/70 success (ACR exact 95% interval: 87.98–99.11%). ACR reused the scene block on 25.24% of queries and reduced wall time by 3.96% relative to the historical sequential FR path. Against BFR, however, it reduced visual CUDA time by only 8.46% and increased complete wall time from 1188.28 to 1190.97 ms/query. Both efficiency gates failed.

An intermediate V4 audit exposed another controller error. A nominal horizon of two still permitted reuse streaks of length two. Its closest candidate reached 35.76% scene reuse and a predicted 12.15% visual reduction, but only a 0.17% predicted wall-time gain over BFR. V5 added an explicit post-reuse refresh latch. The corrected isolated-reuse controller passed an outcome-blind offline screen at 35.48% reuse and 17.74% logical visual-work reduction with maximum reuse streak one.

The real-tensor CUDA-graph implementation did not produce a valid timing or correctness result. The final V10 attempt stopped during its first downstream graph capture with 7.62 GiB of remaining memory headroom. This ruled out insufficient free memory as the immediate explanation. It did not measure method performance.

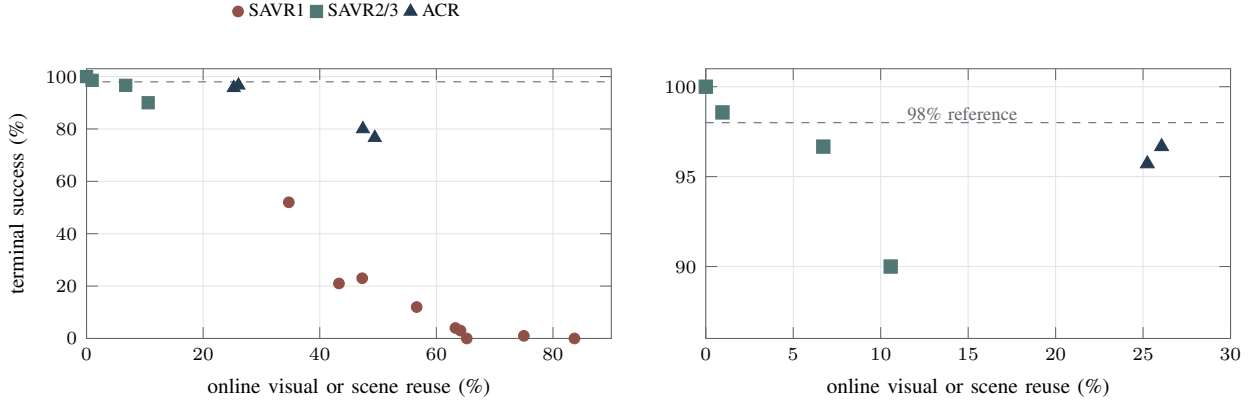


Fig. 2. Observed success–reuse operating points. The right panel enlarges the high-success region without moving points. SAVR1 uses 100 episodes per setting, SAVR2 uses 30, SAVR3 uses 70, ACR A5 uses 30 per setting, and ACR V3 uses 70. The plot is descriptive because stages used different populations and gates. Intervals are reported in tables and text.

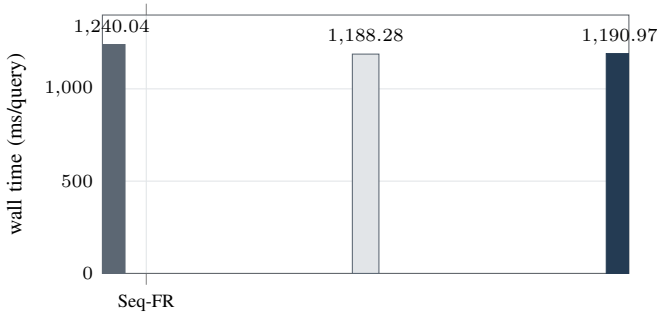


Fig. 3. Closed-loop ACR V3-D wall time. ACR improved over sequential FR but did not improve on the stronger batched full-refresh baseline.

C. BRACE: Fine-grained reuse exposed action drift

BRACE B3 completed 356 physical model queries. Table VI shows the four profiles. P2-D50 reduced the accelerated-query segment by 18.71% and the complete contract cycle by 12.23%. It passed both speed thresholds. Each profile included 42 timed cached actions, for 168 comparisons in total. All 168 differed numerically from their dense custom-path actions, while all gripper decisions matched. B3 therefore stopped before closed-loop evaluation.

This result supports an internal speed–distortion observation, but its official-policy interpretation is provisional because the later audit found the one-token action-state shift. It should not be reported as a faithful OpenVLA-OFT action comparison without re-execution under the corrected helper.

D. PAIR: The development risk signal did not confirm

PAIR P3R vectorized all actual-source tile comparisons. Across 210 model queries, all four profile/horizon timing points passed. D62 at horizon four achieved 24.59% raw saving, a 24.40% one-sided lower bound, a 17.05% conservative net lower bound, and 0.78% routing overhead.

The initial 40-contract P4 development study was encouraging: structured Spearman correlation was 0.3452, matched-service CVaR_{0.9} improvement was 21.48%, and horizon-two/four improvement was 25.85%. The lower confidence

bounds failed, so a separately frozen 240-contract P4B confirmation was run. P4B weakened rather than merely widened the result: Spearman correlation was 0.2001 (one-sided lower bound 0.0999), matched-service CVaR_{0.9} improvement was 3.14% (lower bound -1.07%), and horizon-two/four improvement was 1.62%. All primary reliability gates failed.

E. Corrected CAC: Systems feasibility passed, substrate viability failed

After semantic correction, CAC C1 requalification completed all 97 planned model calls. The corrected custom dense path matched the official evaluator exactly. The D62 horizon-four path retained 22.5969% median gross complete-cycle saving. Feature extraction required 10.5964 ms median, the full untrained adapter forward pass required 2.3883 ms, and peak aggregate memory was 16,789 MiB. These measurements establish that the adapter architecture fit the available hardware and left timing headroom. They do not establish correction quality.

C1H then opened the prespecified closed-loop headroom screen. Exactly 120 paired conditions produced 240 terminal episodes and 8,525 model queries. Dense inference succeeded in 68/120 episodes (56.67%), whereas corrected D62 succeeded in 0/120. D62 failed all 30 episodes in each suite. The independent analyzer passed all 12 execution and reconciliation checks.

All three frozen stop conditions triggered. Dense success was below 75%. D62 success was below 50%. The dense-minus-D62 gap exceeded 35 pp. Therefore, the extension was not opened and the adapter was not trained. The result rejects D62 as the planned CAC substrate. It does not establish that every learned corrector or cache-aware VLA must fail.

VII. CROSS-METHOD ANALYSIS

A. A recurring reliability–efficiency boundary

Figure 2 shows the coarse operating-point pattern. More permissive SAVR and ACR settings increased reuse while reducing observed success. Strong safeguards recovered success

TABLE VI
BRACE TIMING/PARITY AND PAIR INDEPENDENT-CONFIRMATION RESULTS AS ORIGINALLY RECORDED. BRACE AND PAIR
OFFICIAL-POLICY-DEPENDENT INTERPRETATIONS ARE PROVISIONAL AFTER THE SEMANTIC AUDIT.

Program	Profile or endpoint	Point estimate	Required	Lower bound	Result
BRACE B3	P1-S25 complete-cycle reduction	4.82%	$\geq 8\%$	–	Speed fail, action parity 0/42
	P1-S50 complete-cycle reduction	7.68%	$\geq 8\%$	–	Speed fail, action parity 0/42
	P2-D25 complete-cycle reduction	7.30%	$\geq 8\%$	–	Speed fail, action parity 0/42
	P2-D50 complete-cycle reduction	12.23%	$\geq 8\%$	–	Speed pass, action parity 0/42
PAIR P4B	Structured Spearman	0.2001	≥ 0.30	0.0999 (> 0.15 req.)	Fail
	Matched-service CVaR _{0.9} improvement	3.14%	$\geq 15\%$	–1.07% (> 0 req.)	Fail
	Horizon-two/four CVaR _{0.9} improvement	1.62%	$\geq 10\%$	–	Fail

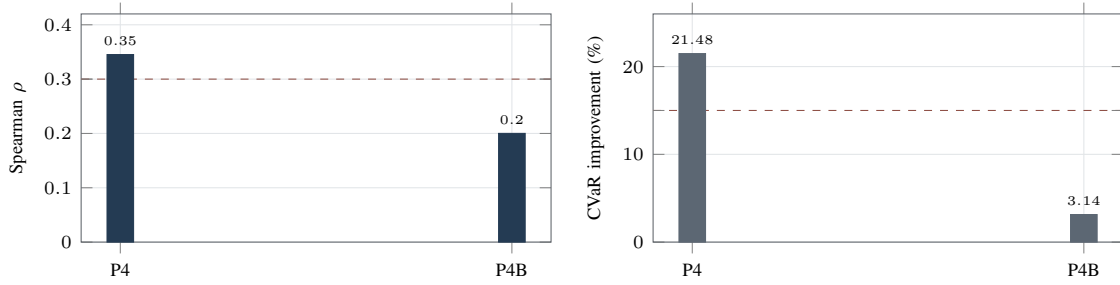


Fig. 4. PAIR development (P4) and independent confirmation (P4B). Dashed lines are the primary point-estimate requirements. P4B did not confirm the initial signal.

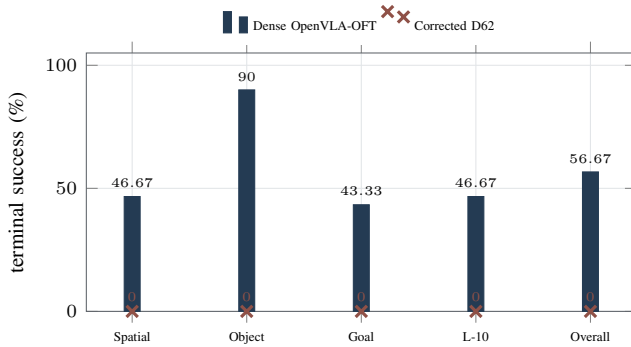


Fig. 5. Corrected C1H closed-loop success. Each suite contains 30 episodes per arm. “Overall” contains 120. The dense arm also missed its qualification threshold, which limits claims about absolute policy performance. D62’s 0/120 result makes it ineligible for CAC training.

TABLE VII
CORRECTED C1H OUTCOMES WITH EXACT TWO-SIDED 95% BINOMIAL INTERVALS.

Suite	Dense (95% CI)	D62 (95% CI)	Gap
Spatial	14/30 (28.34–65.67)	0/30 (0–11.57)	46.67 pp
Object	27/30 (73.47–97.89)	0/30 (0–11.57)	90.00 pp
Goal	13/30 (25.46–62.57)	0/30 (0–11.57)	43.33 pp
LIBERO-10	14/30 (28.34–65.67)	0/30 (0–11.57)	46.67 pp
Overall	68/120 (47.31–65.68)	0/120 (0–3.03)	56.67 pp

only by suppressing reuse. ACR V3 is the important qualification: camera asymmetry retained success at a nontrivial scene-reuse rate, but the gain disappeared against BFR. Thus, the central conflict was not simply “reuse always fails.” It was that useful *incremental* speed and validated reliability did not coexist under the tested boundaries.

B. Why mixed-age K/V is difficult to repair

The decoder cache is not a collection of independent image descriptors. At each layer, visual, instruction, proprioceptive, and action-placeholder states interact. Equation (5) can therefore combine groups that encode different physical times and different upstream histories. Recursive reuse compounds this mismatch because a source state may itself have been formed from older sources.

CAC assumed that the downstream effect of this high-dimensional error could be represented as a low-dimensional action correction. This remains possible for mildly degraded substrates. C1H shows that D62 was not mildly degraded under the tested closed-loop conditions. It lost every episode. Training a 5.07-million-parameter residual model on that base would no longer test the intended small-correction regime. It would also weaken the planned efficiency claim.

C. Offline proxies were useful but insufficient

SAVR dense-trajectory replay estimated thresholds but did not predict online reuse. BRACE action parity detected numerical drift but did not measure task consequence. PAIR’s intervention labels were closer to the mechanism, yet the risk ranking and tail improvement weakened on an independent population. These findings support a staged validation rule: use offline metrics to reject clearly poor designs, but open a small paired closed-loop screen before large-scale optimization or confirmatory investment.

D. The dense baseline must be optimized

ACR V3 improved on sequential FR but not on BFR. The distinction matters because batching removes redundant framework overhead without introducing stale information.

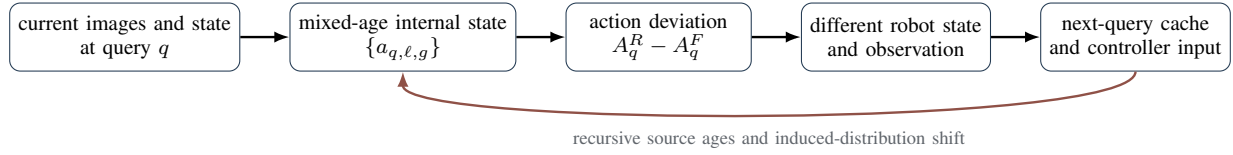


Fig. 6. Mechanism connecting temporal reuse to closed-loop failure. Reuse changes a high-dimensional internal state. The resulting action chunk changes the observations on which later cache decisions are made. Offline dense-trajectory replay omits this feedback.

Any future cache method should first compare against batching, compiler optimization, quantization, and other fresh-information baselines available on the same model. A cache speedup relative to an unnecessarily slow dense path is not an incremental method result.

VIII. VALIDITY, FAILURES, AND REPRODUCIBILITY

A. Engineering failures do not determine method performance

The project experienced repeated technical stops, including incorrect low-level invocation counts, serialized audit work on the critical path, CUDA-graph capture failures, cache-position incompatibilities, loader/normalization mismatches, output-schema errors, and the shifted action readout. Table VIII records their scientific treatment. None of these incomplete runs contributes a terminal-success or final timing estimate.

The semantic audit also exposes a general testing hazard. If candidate and reference paths share the same indexing or preprocessing code, exact agreement only establishes internal consistency. At least one test must compare the final model boundary with an independently implemented or released oracle.

B. Evidence preservation

The repository retains controller and cache code, frozen configurations, schemas, analysis scripts, decision records, machine-readable summaries, and full report hashes. The principal SAVR records are PHASE6_CALIBRATION, PHASE6R_D_STAGE1, and PHASE6S_D_VALIDATION. The principal ACR records are PHASE_A5, PHASE_V3_C, and PHASE_V3_D. BRACE and PAIR are recorded in BRACE_B3_V05, PAIR_P3R, and PAIR_P4B. The correction record is OPENVLA_CUSTOM_PATH_SEMANTIC_AUDIT, together with the S3, S4, and S5 requalification reports. The corrected CAC records are CAC_C1_S5_REQUALIFICATION_PASS and CAC_C1H_S6_RECOVERY01_GATE_H_STOP.

Failed episodes were retained. Gates were not relaxed after outcome access. Technical recoveries corrected bounded implementation defects without reassigning failed scientific populations. The repository is maintained at <https://github.com/vwdr/SAVR>. Large immutable rollout artifacts remain project-local pending an archival release.

IX. LIMITATIONS

The experiments use one OpenVLA-OFT checkpoint, one simulator benchmark family, one hardware class, and phase-specific fixed seeds or state schedules. Generalization to other

VLA architectures, action heads, camera geometries, and real robots is unknown. Not every external acceleration method was reproduced on the same checkpoint and hardware. The related-work comparison therefore distinguishes mechanisms but does not establish numerical superiority.

The study is sequential. SAVR, ACR, BRACE, PAIR, and CAC use different populations, outcomes, and gates. Cross-method plots summarize the observed program. They are not randomized head-to-head comparisons. The exact binomial intervals also omit dependence among tasks and initial states.

Dense C1H success was 56.67%. This was below its 75% gate and the published OpenVLA-OFT four-suite average [4]. This limits inference about absolute checkpoint quality in that run. It does not convert D62's 0/120 result into evidence for training CAC. CAC was not trained. We reject one corrected D62 substrate as the planned CAC base, not residual correction or learned cache-aware models as a class.

BRACE and PAIR were executed before the independent readout audit. Their official-policy-dependent action and risk claims are provisional. The corrected CAC systems and C1H results supersede their role as evidence for D62 feasibility. The final protected holdouts for earlier programs remained sealed after development gates failed. This prevents post-hoc rescue but limits confirmatory population-level claims.

X. IMPLICATIONS FOR FUTURE WORK

The combined evidence does not justify another threshold revision or post-hoc action corrector on D62. It supports two different directions.

First, future work can reduce the cost of *fresh* information: current-frame token selection or merging, adaptive spatial resolution, quantization, batching, and compiler-aware execution. These methods avoid recursive source age and should be evaluated against an optimized dense baseline.

Second, if temporal reuse remains necessary, the model should be trained under the actual reuse process so that robust representations are formed before the action head. A learned selector or internal adapter should receive exact source provenance and should be exposed during training to multi-query recursive states, not only isolated token perturbations. The earliest expensive milestone should be a small paired closed-loop viability screen. Systems optimization and large offline studies should follow, not precede, evidence that the modified policy can still control the robot.

XI. CONCLUSION

This study followed temporal visual reuse from a simple state-aware whole-prefix gate to camera-specific refresh, fine-grained K/V reuse, learned risk routing, and residual action

TABLE VIII
REPRESENTATIVE TECHNICAL FAILURES AND THEIR EVIDENTIARY TREATMENT.

Failure class	Consequence	Resolution or containment	Scientific status
ACR V2 critical-path audits	Visual CUDA work fell, but wall time increased by about 41%	Removed serialization from production path. Introduced BFR and V3 paired timing	V2 latency failure retained
ACR V5 CUDA graphs	No valid correctness or timing record. Final capture failed with memory headroom	Preserved V01–V10 technical records. Made no task claim	Technical stop only
BRACE/PAIR/CAC tail slice	Custom action states shifted one token from official evaluator	Added one structural readout contract and an independent official oracle. Requalified CAC exactly	Historical official-policy claims provisional
CAC C1H integration/schema stops	Runs ended before complete terminal population	Used frozen fail-closed recoveries. No partial outcomes entered analysis	Excluded from C1H estimate
Corrected C1H	Completed 240 terminal episodes. Every reconciliation check passed	Independent analysis reproduced the stop decision	Valid scientific stop

correction. The methods repeatedly exposed the same boundary. Reuse could remove meaningful computation, but operating points with demonstrated savings changed policy behavior. Safeguards that restored reliability removed the savings or lost to a stronger fresh baseline.

The corrected C1H experiment provides the clearest endpoint. D62 retained measured systems headroom and exact official-policy boundary checks, yet achieved 0/120 terminal successes against 68/120 for dense inference. Under the declared CAC design, this substrate was not recoverable enough to justify adapter training. The scientifically supported conclusion is therefore bounded: recursively mixed-age temporal reuse did not yield a reliable and useful operating point for this OpenVLA-OFT/LIBERO configuration. Future positive work should preserve current task-critical information or learn robustness during representation formation, with independent semantic checks and early closed-loop evaluation.

REFERENCES

- [1] A. Brohan *et al.*, “RT-1: Robotics transformer for real-world control at scale,” in *Proc. Robotics: Science and Systems*, 2023.
- [2] B. Zitkovich *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *Proc. Conf. Robot Learning*, ser. Proc. Mach. Learn. Res., vol. 229, 2023, pp. 2165–2183.
- [3] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “OpenVLA: An open-source vision-language-action model,” in *Proc. Conf. Robot Learning*, ser. Proc. Mach. Learn. Res., vol. 270, 2025, pp. 2679–2713.
- [4] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” in *Proc. Robotics: Science and Systems*, 2025.
- [5] S. Xu, Y. Wang, C. Xia, D. Zhu, T. Huang, and C. Xu, “VLA-Cache: Efficient vision-language-action manipulation via adaptive token caching,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025, pp. 164 448–164 473.
- [6] Y. Yang, Y. Wang, Z. Wen, L. Zhongwei, C. Zou, Z. Zhang, C. Wen, and L. Zhang, “EfficientVLA: Training-free acceleration and compression for vision-language-action models,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025, pp. 40 891–40 914.
- [7] Y. Li, L. Ma, H. Ding, and L. Zhu, “DepthCache: Depth-guided training-free visual token merging for vision-language-action model inference,” *arXiv preprint arXiv:2603.10469*, 2026.
- [8] Y. Wei, J. Fan, J. Guo, R. Zhen, R. Shao, X. Su, Z. Xie, H. Yao, and S. Yang, “Learning to accelerate vision-language-action models through adaptive visual token caching,” *arXiv preprint arXiv:2602.00686*, 2026.
- [9] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 44 776–44 791.
- [10] Z. Wu, K. Kawaharazuka, and K. Okada, “Neural introspection gating for adaptive KV-cache reuse in vision-language-action models,” *arXiv preprint arXiv:2608.10824*, 2026.
- [11] Z. Li, R. Zhao, J. Wang, H. Zhu, C. Zhang, G. Valenzise, W. Lin, and K.-M. Lam, “Just noticeable difference modeling for token compression in vision-language-action models,” *arXiv preprint arXiv:2608.21247*, 2026.
- [12] W. Yu, T. Wang, F. Li, J. Li, and L. Zhu, “AC²-VLA: Action-context-aware adaptive computation in vision-language-action models for efficient robotic manipulation,” *arXiv preprint arXiv:2601.19634*, 2026.
- [13] K. Sendai, M. Alvarez, T. Matsushima, Y. Matsuo, and Y. Iwasawa, “Leave no observation behind: Real-time correction for VLA action chunks,” *arXiv preprint arXiv:2509.23224*, 2025.
- [14] Y. Liu, Y. Li, Z. Tang, Y. Zheng, Y. Lin, Q. Wang, Y. Li, S. Liu, S. Zhang, T. Jing, D. Gao, N. Bi, J. Sun, Y. Chen, and H. Li, “Latent bridge: Feature delta prediction for efficient dual-system vision-language-action model inference,” *arXiv preprint arXiv:2605.02739*, 2026.
- [15] S. Ross, G. Gordon, and J. A. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proc. Int. Conf. Artificial Intelligence and Statistics*, ser. Proc. Mach. Learn. Res., vol. 15, 2011, pp. 627–635.
- [16] C. J. Clopper and E. S. Pearson, “The use of confidence or fiducial limits illustrated in the case of the binomial,” *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934.